

RuYi: Optimizing Burst Buffer Through Automated, Fine-Grained Process-to-BB Mapping

Yusheng Hua¹, Xuanhua Shi¹, Senior Member, IEEE, Ligang He², Member, IEEE,
Kang He¹, Teng Zhang¹, Hai Jin¹, and Yong Chen³, Member, IEEE

Abstract—Current supercomputers use an SSD-based storage layer called Burst Buffer (BB) to provide I/O-intensive applications with accelerated storage access. However, efficiently utilizing this limited and expensive storage remains a critical issue, creating an urgent need for implementing Quality of Service (QoS) in BB. To address this, we propose RuYi, a QoS-aware method to provide applications with bandwidth guarantees in the BB file system. RuYi tackles two main issues. First, it quantitatively profiles available bandwidth resources in BB to ensure reliable QoS, a crucial aspect seldom studied in the literature. Second, RuYi offers fine-grained process-level QoS via an innovative process-to-BB mapping, maximizing resource utilization—something not achievable with conventional coarse-grained compute-to-BB mapping. We evaluated RuYi on a subsystem of the leading exascale supercomputer Sunway, consisting of 4,000 compute nodes and 200 BB nodes. The experimental results demonstrate that RuYi achieves an impressive end-to-end bandwidth control accuracy of 97%, while improving BB utilization by up to 116% compared to conventional coarse-grained compute-to-BB mapping.

Index Terms—HPC, burst buffer, file system, QoS.

I. INTRODUCTION

BURST Buffer (BB for short) [1] serves as an intermediate storage layer that sits between the compute nodes and the underlying parallel file system (PFS). Leveraging high-speed storage media such as SSDs, BB effectively absorbs bursty I/O operations and allows the applications to quickly return to compute stages. This in turn enhances the overall performance and reduces core usage hours [2]. BB has recently gained widespread adoption in high performance computing systems,

scaling from petascale platforms such as Cori and Trinity to exascale systems such as Aurora and the new-generation Sunway.

Despite BB's appealing performance characteristics, its high cost makes it a scarce resource [3]. Therefore, sharing limited BB resources among multiple applications is inevitable. In this shared scenario, applications are susceptible to performance degradation due to interference from other applications. To achieve optimized BB allocation, existing research studies propose either interference-aware methods [4], [5], [6], [7], [8], [9] or QoS-aware methods [10], [11], [12], [13]. The former methods mainly detect inter-application interference and utilize efficient I/O sharing policies to maximize overall bandwidth, while the latter methods prioritize ensuring the bandwidth requirements of individual applications via resource reservation.

The dilemma between guaranteeing the bandwidth requirements of individual applications and maximizing overall BB utilization stems from two critical issues. First, existing methods lack the ability to quantitatively profile the available bandwidth on BB nodes. This capability is crucial and forms the foundation of almost all scheduling strategies. Without it, multiple applications can easily be allocated to BB nodes with insufficient resources, forcing them to share the limited resources. Second, current HPC systems typically adopt a compute-to-BB mapping where one compute node can be bound to only one BB node. The coarse-grained nature of the compute-to-BB mapping relied upon by most methods hinders fine-grained and flexible BB resource scheduling. For example, when a BB node has limited remaining resources, bandwidth demands at the granularity of compute nodes often cannot be fulfilled, resulting in wastage of resources on the BB node. These two critical issues motivate us to reexamine the design of the BB allocation method.

RuYi is a BB allocation mechanism designed for BB file systems to offer reliable and fine-grained I/O QoS. **To provide reliable QoS**, RuYi abstracts and maintains a dynamic bandwidth pool to quantitatively profile the available bandwidth resources in BB nodes. Specifically, since the consumption of SSD bandwidth resources fluctuates with the I/O characteristics of applications, RuYi requires three parameters, operation type (read or write), request size, and desired bandwidth, to conduct the profiling. **To provide process-level QoS**, RuYi adopts the process-to-BB mapping, allowing I/O processes of a compute node to be bound to different BB nodes. An I/O process refers to a process (such as a regular MPI process) that

Received 20 August 2024; revised 20 October 2024; accepted 19 November 2024. Date of publication 2 December 2024; date of current version 12 February 2025. This work was supported in part by the Major Program (JD) of Hubei Province under Grant 2023BAA024. Recommended for acceptance by J. Zhai. (Corresponding author: Xuanhua Shi.)

Yusheng Hua, Xuanhua Shi, Kang He, Teng Zhang, and Hai Jin are with the National Engineering Research Center for Big Data Technology and System, Services Computing Technology and System Lab, Huazhong University of Science and Technology, Wuhan 430074, China (e-mail: yshua@hust.edu.cn; xhshi@hust.edu.cn; kanghe@hust.edu.cn; tengzhang@hust.edu.cn; hjin@hust.edu.cn).

Ligang He is with the University of Warwick, CV4 7AL Coventry, U.K. (e-mail: ligang.he@warwick.ac.uk).

Yong Chen is with Texas Tech University, Lubbock, TX 79409 USA (e-mail: yong.chen@ttu.edu).

Digital Object Identifier 10.1109/TC.2024.3510624

also performs I/O operations. Based on this mapping strategy, the bandwidth requirement can be considered at the granularity of individual I/O processes. Given that parallel HPC applications are generally comprised of numerous I/O processes, this fine-grained process-to-BB mapping naturally breaks down the otherwise coarse-grained bandwidth requirements. With these fine-grained bandwidth requirements, the resources fragments in BB will have more opportunities to be utilized, thus improving the overall resource utilization. Furthermore, the process-level QoS can also alleviates straggler issues [11], [14], where multiple I/O processes of an application receive different I/O services, leading to asynchronous I/O progress. However, this significant shift in mapping granularity, from compute-to-BB mapping to process-to-BB mapping, necessitates comprehensive changes to the software stack. RuYi optimizes several key components to achieve this goal including I/O interfaces, BB resource manager, and the I/O scheduler in the filesystem server-side.

RuYi is built on HadaFS [15], a novel BB file system designed for the leading supercomputer Sunway. Performance evaluation conducted on a of Sunway sub-system, consisting of 4,000 compute nodes and 200 BB nodes, show that RuYi achieves an impressive end-to-end bandwidth control accuracy of 97%. Additionally, RuYi improves BB utilization by a range of 18%-116% when compared to the allocation method that adopts compute-to-BB mapping. Moreover, RuYi exhibits very small overhead in exascale systems. Specifically, the one-time startup takes at most 35 seconds for the entire Sunway supercomputer, which consists of 600,000 I/O processes and 600 BB nodes. In more typical scenarios, RuYi incurs a startup overhead of less than 2 seconds and maintains runtime overhead at the microsecond level.

The contributions of this paper are summarized as follows:

- We propose a QoS-aware BB allocation method that provides reliable QoS which is essential for resource scheduling but seldom studied.
- We design and implement a novel fine-grained process-to-BB mapping, in which BB resources are allocated to individual I/O processes, improving resource utilization and ensures synchronous I/O progress.
- We conducted evaluation experiments on the leading supercomputer Sunway. The results demonstrate that with a small overhead, RuYi can provide an accurate end-to-end bandwidth control (with the accuracy of 97%) and improve the utilization of BB resources significantly (by up to 116%).

II. BACKGROUND AND MOTIVATION

A. Lack of Efficient BB Allocation System

Limited by the high cost of SSDs, BB is a scarce resource in an HPC architecture. In Sunway, the ratio of the number of compute nodes to that of BB nodes is around 170:1 [7], [15]. As some of the BB nodes are reserved and not for daily uses, the actual ratio can reach as high as 512:1. Sharing the limited BB resources properly among concurrent applications is a critical issue.

Inter-application interference is a pivotal factor affecting I/O performance and user experience [16]. Consequently, many research studies emphasize contention-aware scheduling strategies. Datawarp [17], a classic example deployed on Cori, employs an interference policy where each BB node is occupied by only one job at a time, eliminating inter-application interference. Newer systems like DFRA [7] and CARS [5] are less restrictive, allowing different applications to share BB nodes when the interference factor is low. However, due to the performance degradation experienced by most applications when sharing BB, current methods tend to allocate exclusive nodes to jobs to ensure application performance. This leads to the problem of resource over-allocation. A load summary in Sunway also revealed that over 90% of BB nodes operate below 50% of their peak bandwidth [18], indicating significant under-utilization. This statistical data does not mean that BB resources are sufficient. Rather, the underutilization is caused by the current way in which the BB nodes are managed. Even when more applications are running in the supercomputer, more BB nodes need to be deployed, resulting in low utilization of BB nodes on average in the system. With better management, the same number of BB nodes can actually serve more applications, leading to improved utilization of BB nodes, which is the objective of our work. This is also the reason why designing a highly efficient BB management system is a widely recognized need. Not only Sunway, but also Summit and Cori have raised the same need [19].

B. Challenges and Opportunities of Introducing QoS Into BB

In BB resource allocation, the primary objective is to ensure the I/O performance of applications. A logical approach is to incorporate the QoS criteria into BB to enforce bandwidth guarantees. While this concept is not novel and has been explored with considerable success in various research studies, existing approaches mainly leverage QoS algorithms to manage application bandwidth on the server-side. Typically, they do not consider the mapping between compute resources and BB. In this setup, the QoS control algorithm can exhibit limitations. The most apparent is that, if bandwidth resources on pre-allocated BB nodes are inadequate, applications may suffer performance degradation. Consequently, we argue that exclusively depending on server-side QoS control is insufficient. Considering physical resource mapping when defining QoS policies is crucial, though more complex. A significant challenge is ensuring sufficient bandwidth resources to satisfy QoS demands. To our knowledge, this aspect, though vital for QoS, has been largely overlooked in existing literature.

To tackle this challenge, abstracting bandwidth resources on individual BB nodes is essential. Achieving this on HDD-based systems is extremely difficult due to their unstable performance, especially with random data access patterns. Conversely, SSDs, being flash-based storage devices, deliver more consistent and predictable performance, facilitating bandwidth resource abstraction.

It is worth noting that the amount of SSD bandwidth resources consumed by an I/O load is primarily determined by

the I/O operation type (read or write), the I/O request size, and the bandwidth requirement of the I/O load. Therefore, when assessing the available bandwidth resources of an SSD, it is pivotal to account for these parameters. In our current system design, users or administrators will provide these parameters when interacting with the QoS interface. While this may impose additional responsibilities on them, it considerably simplifies system design. Dynamically detecting and setting these parameters in real-time would complicate resource allocation logic. Nevertheless, thanks to advancements in performance analysis tools [18], [20] and deeper insights into I/O behaviors in HPC systems [21], [22], users and administrators can determine the specific I/O demands of distinct HPC tasks, and even the needs of individual I/O stages [23]. This empowers them to state their QoS needs and supply them as inputs to the QoS system.

C. Advantages of Process-to-BB Mapping

Besides ensuring application performance, optimizing the limited BB resources to accommodate more users is vital. A common strategy to enhance resource utilization in system design is to reduce resource allocation granularity. From our study of HPC applications and storage systems, the I/O process stands out as a favorable choice over the application or compute node, for three primary reasons.

First, I/O processes serve as the primary units that generate I/O operations in HPC applications. Moreover, an HPC application typically encompasses multiple I/O processes. This allows process-to-BB mapping to partition an application's bandwidth requirements into smaller segments. Second, the process-to-BB mapping introduces more adaptive scheduling strategies. For instance, if we want to use as many BB nodes as possible for the sake of SSD wear balancing, this mapping can distribute the load over more nodes. In theory, process-to-BB mapping allows one I/O process link to one BB node. In contrast, under compute-to-BB mapping, all the I/O processes running on a compute node can connect only to the single BB node allocated to that compute node. Lastly, process-to-BB mapping can effectively tackle the straggler issue [11], [14] by allocating bandwidth resources uniformly among different I/O processes within an application.

The biggest difference between the compute-to-BB mapping and the process-to-BB mapping is that the process-to-BB mapping allows different I/O processes running on the same compute node to be mapped to different BB nodes. In contrast, in the compute-to-BB mapping, all the I/O processes running on the same compute node can only be bound to a single BB node.

We use an example below to illustrate the restriction of the conventional compute-to-BB mapping and the advantage of our process-to-BB mapping. In Fig. 1, we assume two jobs, each with 8 processes running on 2 Compute Nodes (denoted by CN in the figure). Job 1 has the I/O pattern of N-1 (N processes access a common shared file), while Job 2 has the N-N pattern (N processes access N separate files). The bandwidth requirements of Job 1 and Job 2 are 2GB/s and 4GB/s, respectively.

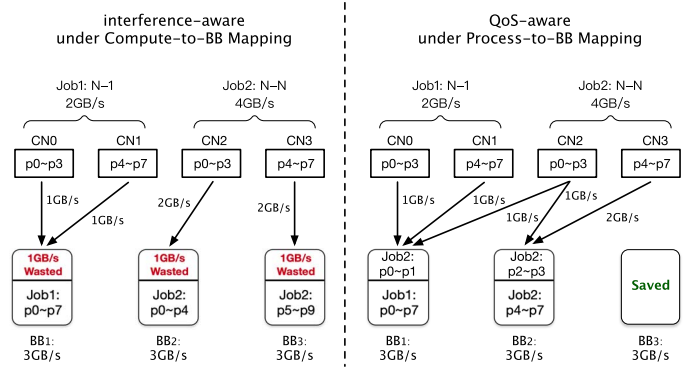


Fig. 1. Comparison between interference-aware BB allocation under compute-to-BB mapping and qos-aware BB allocation under process-to-BB mapping. With the help of QoS and process-to-BB mapping, bandwidth requirements can be guaranteed at a finer granularity in shared scenarios, reducing resource waste.

The shared file accessed by Job 1 resides on the first BB node (BB1), and therefore the 2 compute nodes of Job 1 forward I/O requests to BB1.

Fig. 1 (left) depicts the scheduling result of the interference-aware BB allocation methods under the compute-to-BB mapping. This mapping strategy restricts that all the processes on one computing node can only be bound to one BB node. The interference-aware methods detect potential interference and decide whether two jobs should share a BB node. If the I/O requests from CN2 were forwarded to BB1, CN2 may be allocated with only 1GB/s bandwidth because the number of processes from Job 2 (i.e., 4 in this example) is only half of the number of processes from Job 1 (i.e., 8). This will result in performance degradation for Job 2 (Job 2's processes running on CN2 requires the bandwidth of 2GB/s). Therefore, the interference-aware methods tend to bound CN2 to a separate BB node. Consequently, 3 BB nodes will be allocated in total for these two jobs, which will result in the 1GB/s bandwidth waste on each BB node.

The right of Fig. 1 shows the advantages of our QoS-aware method under the Process-to-BB mapping. With the ability of process-to-BB mapping, the 2GB/s bandwidth requirement of CN2 can be divided into two parts, and Processes P0 and P1 are mapped to BB1 while P2 and P3 are mapped to BB2. Consequently, Job 1 and Job 2 can share the bandwidth of BB1 in harmony. Similarly, processes P4 to P7 of Job2 can also be mapped to BB2. With our process-to-BB mapping, the bandwidth of BB1 and BB2 are fully utilized and BB3 can be spared. Note that the pure QoS-aware methods without the support of process-to-BB mapping cannot achieve such fine-grained allocation results. When the remaining bandwidth on a busy BB node (like BB1 in this example) is not enough for a compute node (like CN2), the pure QoS-aware methods cannot make use of the remaining bandwidth. This is the case for all the existing methods, not just for a particular system.

However, this significant shift in mapping granularity, from compute-to-BB mapping to process-to-BB mapping,

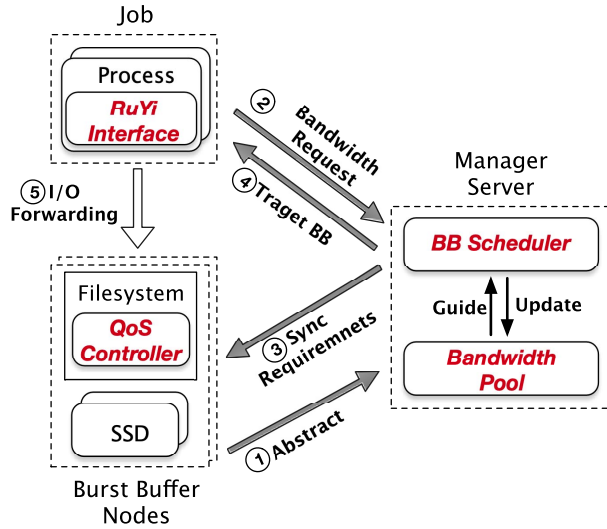


Fig. 2. System overview of RuYi.

necessitates comprehensive changes to the software stack in the file system, which we will delve into in the subsequent sections.

III. DESIGN OF RUYI

In this section, we first present an overview of RuYi and then introduce the designs in detail. Last but not the least, we discuss usage scenes and limitations of RuYi.

A. System Overview

As shown in Fig. 2, RuYi has four components: Bandwidth Pool, RuYi Interface, BB Scheduler, QoS Controller. The I/O flow of RuYi can be described as follows. First, RuYi abstracts a bandwidth pool from all the SSDs in BB nodes to profile the overall available bandwidth resources. Second, I/O processes of an HPC job use RuYi interfaces to send bandwidth requests to the BB scheduler. Third, the BB scheduler finds proper BB nodes that have sufficient bandwidth to satisfy the received bandwidth requests based on the bandwidth pool. Then, it synchronizes the bandwidth requirements with the selected BB nodes to prepare them for upcoming I/O requests. Fourth, the BB scheduler returns the target BB nodes to the I/O processes. Fifth, I/O processes dynamically bind to the target BB nodes and start to send I/O requests. Finally, these I/O requests are scheduled by the file system server residing on each BB node according to their previously synchronized bandwidth requirements. Specifically, a component called the QoS controller uses the mClock [24] algorithm to ensure bandwidth guarantees.

B. Bandwidth Pool

Profiling the available bandwidth resources is the basis of providing reliable bandwidth QoS. In this section, we introduce the data structure of the bandwidth pool and the method to predict available bandwidth.

Data structure. To make the bandwidth pool load-aware, we use a load list that keeps track of the real-time I/O information

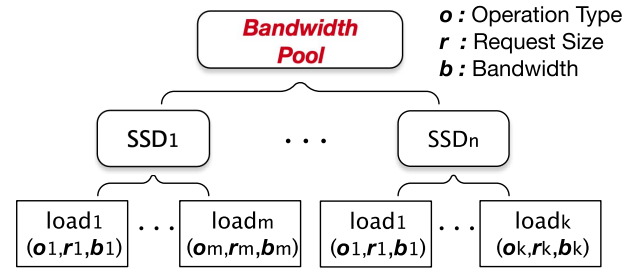


Fig. 3. Organization of bandwidth pool.

TABLE I
INPUT AND OUTPUT OF F_b

Input		Output
o write: 0 read: 1	r in KB	Load List in $[(o, r, b), \dots]$
0	1024	$b_{(o,r)}$ in MB/s
		891.3

of all the loads served by each SSD. As shown in Fig. 3, the run-time I/O information of a load is described with three variables: 1. o , which refers to the operation type (read or write); 2. r , which refers to the request size; 3. b , which refers to the desired bandwidth. Choosing these three variables has two primary considerations. First, they are the prominent I/O characteristics of an HPC application. Second, they determine the performance of an SSD.

It is noted that loads with the same (o, r) values can be merged into a single (o, r, b) tuple by accumulating their desired bandwidth b . For instance, two tuples of (write, 4KB, 10MB/s) can be merged into a single tuple of (write, 4KB, 20MB/s). This merging of loads is necessary as an SSD may serve hundreds of I/O processes that create loads with the same (o, r, b) tuple that belong to a single application. Merging these loads simplifies the maintenance of the bandwidth pool and reduces the computational overhead of subsequent bandwidth prediction.

All of the load lists and their corresponding SSDs eventually form a tree-like data structure that is utilized to keep track of the real-time serving status of the bandwidth pool. This structure serves as the basis for all subsequent resource prediction and scheduling.

Prediction method. The prediction refers to deriving the maximum bandwidth that a request of (o, r) can obtain from an SSD with an existing load list. We denote this maximum bandwidth as $b_{(o,r)}$. The prediction is made when the bandwidth pool needs to check whether an SSD can satisfy the bandwidth requirements of an I/O process. In other words, whether an (o, r, b) tuple can be inserted into the load list of an SSD. If $b_{(o,r)} \geq b$, we can insert a load into the list, otherwise, we need to check other SSDs.

We define a function F_b to predict $b_{(o,r)}$. The input and output of F_b are summarized in Table I. As shown in the example in Table I, if the bandwidth requirement of a 1MB write QoS request is no greater than 891.3MB/s, it can be satisfied by an

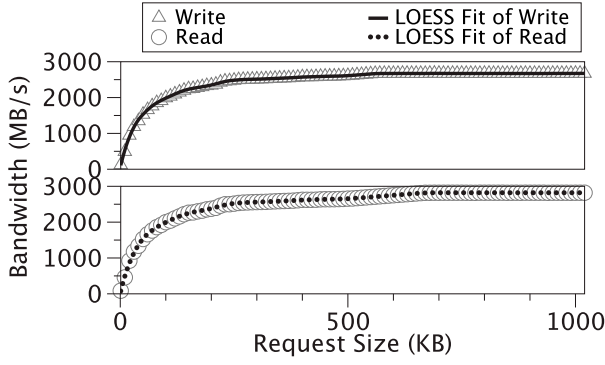


Fig. 4. The relationship between (o, r) and $B_{(o,r)}$, where $B_{(o,r)}$ refers to the maximum bandwidth that a request of (o, r) can get from a spare SSD.

SSD that already has two loads: (0, 1KB, 40MB/s) and (1, 8KB, 150MB/s).

F_b uses an empirical method to get $b_{(o,r)}$, and it has two steps. First, all the loads in a load list are converted into one (o, r, b_{ll}) tuple, where b_{ll} refers to the equivalent bandwidth that a request of (o, r) would receive if it consumes the same amount of SSD resources as all the loads in the load list. Second, $b_{(o,r)}$ is calculated as the difference between $B_{(o,r)}$ and b_{ll} , where $B_{(o,r)}$ refers to the maximum bandwidth that a request of (o, r) can get from a spare SSD.

The proposed bandwidth conversion method is based on the fact that SSDs are not sensitive to the randomness of requests. So the maximum bandwidth that a request can acquire is mainly determined by its I/O size and I/O type. For a given I/O load (o, r, b) with the maximum bandwidth denoted by $B_{(o,r)}$, this I/O load uses $b/B_{(o,r)} * 100\%$ of the maximum bandwidth. The utilization percentage is used as the basis for bandwidth utilization conversion between different I/O loads. For example, if $B_{(0,4KB)}$ is 30MB/s and $B_{(0,8KB)}$ is 60MB/s, an I/O load of (0,4KB,10MB/s) utilizes $10/30 * 100\%$ of $B_{(0,4KB)}$. This I/O load, (0,4KB,10MB/s), can be considered equivalent to an I/O load of (0,8KB,20MB/s) in terms of bandwidth utilization within their respective maximum bandwidths, because $20/60 = 10/30$.

It is obvious that this bandwidth conversion method requires prior knowledge of $B_{(o,r)}$. Fortunately, $B_{(o,r)}$ is easy to get. As shown in Fig. 4, to get $B_{(o,r)}$, we first collect 100 $B_{(0,r)}$ and 100 $B_{(1,r)}$ where r ranges from 1KB to 1024KB, and then use the LOESS function [25] to fit two smoothing curves for read and write operations respectively. With these two curves in Fig. 4, we can get any $B_{(o,r)}$ to complete the conversion from a load list to one (o, r, b_{ll}) . Finally, F_b can be formulated as Equation 1.

$$F_b((o, r, b), LoadList) = B_{(o,r)} - \sum_{(o_i, r_i, b_i) \in LoadList} \left(B_{(o,r)} * \frac{b_i}{B_{(o_i, r_i)}} \right) \quad (1)$$

C. RuYi Interface

In addition to conventional usage that uses job schedulers to apply for BB resources, RuYi interfaces can be called by users

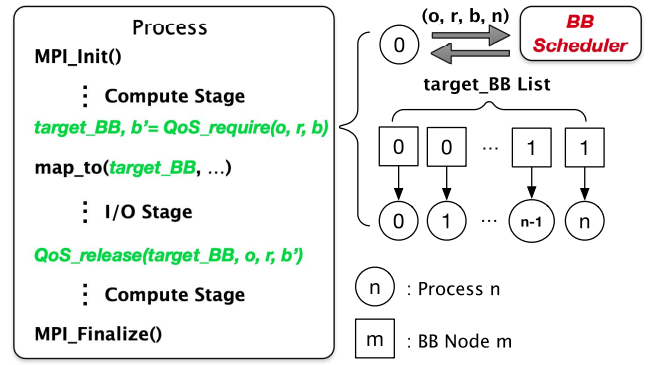


Fig. 5. The interfaces (marked in green italic) provided by RuYi to require and release bandwidth resources.

within their applications. This design decouples I/O management from job schedulers and allows for more flexible resource allocation during runtime.

As shown in Fig. 5, RuYi mainly provides two interfaces, *QoS_require* and *QoS_release*, for users to require and release bandwidth resources from the bandwidth pool, respectively. The ideal position to call these two interfaces is before and after each I/O stage, as it will avoid the resource waste when an application still holds the allocated bandwidth resources even at compute stages. The *QoS_require* interface takes an $(operation_type, request_size, bandwidth)$ tuple as input and returns each process with a target BB node and the actual bandwidth b' . In *QoS_require*, process0 takes charge of raising the QoS request, sending the (o, r, b) tuple along with the number of processes n to the BB scheduler. After receiving a target BB list that contains n target BB nodes, process0 broadcasts the list to the other $n - 1$ processes via inter-process communication. Then, the *map_to()* interface is used to bind a process to its target BB node. *map_to()* is a function provided by HadaFS to bind all the processes of a compute node to a single BB node. We modified this function to allow each process to bind to its own BB node, thereby achieving the process-to-BB mapping. At last, process0 calls the *QoS_release* interface to release the acquired resources.

In real-world scenarios, users may not always receive the desired amount of resources, resulting in $b/n \neq b'$. This is the reason why we use a single process as a proxy to send a QoS request instead of allowing all processes to request resources independently. If we use the latter approach, once a process does not get its desired bandwidth, all the other processes need to request a lower bandwidth again to ensure the inter-process fairness. This will severely complicate the resource allocation procedure and cause extra communication overhead. Our design that uses a single process to request resources largely simplifies the allocation logic and also reduces the burden on the BB scheduler because it does not need to handle thousands of concurrent QoS requests anymore.

In addition to the user-level interface, RuYi also allows administrators to adjust the bandwidth share of a job during runtime. With the command `qos_set -r/w bandwidth -f job_name`, the manager server will first find the target BB

nodes that serve the job and then synchronizes new bandwidth requirements with these BB nodes. It is noted that as the binding between the I/O processes and the BB nodes is performed by user-level `QoS_require` and `map_to`, the adjustment made by administrators during runtime can only change the bandwidth obtained by an I/O process on its current BB node. In other words, if the adjustment range is too large so that an I/O process needs to acquire services from another BB node, the adjustment will not be satisfied. Allowing the BB file system to re-configure the process-to-BB mapping during runtime can solve this issue, but it is hard to implement as it requires complex coordination between the jobs, manager server, and BB nodes. Currently, using the `QoS_require` interface explicitly within an application can proactively re-configure the process-to-BB mapping without disturbing the I/O process, which is a feasible way to freely adjust the bandwidth requirement during runtime. How to change the process-to-BB mapping outside user codes is an interesting issue and will be left for future work.

D. BB Scheduler

The BB scheduler is a central component that communicates with all other components. Upon receiving a QoS request, the BB scheduler first locks the bandwidth pool and searches for target BB nodes capable of satisfying the bandwidth requirements of n processes. It then synchronizes the scheduling results with the target BB nodes, ensuring they are prepared for incoming I/O requests. Finally, it updates the bandwidth pool by inserting corresponding (o, r, b) tuples into the load lists of selected BB nodes and returns the target BB list to process0.

Choosing appropriate BB nodes for a QoS request that includes the bandwidth requirements of n processes is the primary function of the BB scheduler. The scheduling procedure works as follows. The BB scheduler first uses Equation 1 to predict the available bandwidth on each BB node in the bandwidth pool (see section III-B). Then, for each process, the BB scheduler traverses all the BB nodes and finds a BB node capable of satisfying the bandwidth requirement of a process. If all n processes find proper BB nodes, the BB scheduler can finish this round of search and return the result. Otherwise, it has to decrease the bandwidth requirement b and repeats the search. If the bandwidth pool does not have enough resources, users may only receive the maximum available bandwidth or seek assistance from administrators to adjust the QoS metrics of other jobs.

It is worth noting that network topology is another factor that may affect the effectiveness of the QoS control for I/O [26], [27]. Hence, when selecting a suitable BB node, in addition to considering the available bandwidth, RuYi will also prioritize BB nodes that are closer to an I/O process. This approach can greatly minimize the inaccuracies in bandwidth control and prevent performance degradation.

The above is the principle idea of the BB scheduler. In the specific design, since RuYi is built on top of HadaFS, it currently relies on HadaFS for metadata management.

We first briefly introduce how HadaFS manages the metadata. HadaFS uses a hash function, which takes as input a file path and name, to locate the BB node that stores the global metadata of the file. The global metadata of the file specifies all the BB nodes that store the segments of the file. On each BB node, HadaFS uses another local metadata server to record the specific location of the file segments stored on the local storage devices. More details about the metadata management in HadaFS can be found in [15].

Based on the metadata management of HadaFS, RuYi only modifies the BB allocation at runtime for write operations. This is because all write operations in an I/O process are directly forwarded to its bound BB node. If the binding of the I/O process moves to another BB node and write the same file, the metadata of the file will be overwritten by new operations. In other words, an I/O process can write to any BB node without worrying about the metadata. However, a read operation may read the file segments residing on multiple BB nodes. These segments will first transfer to the bound BB node of the I/O process and then return to the client. Then, RuYi will insert the load proportionally into the bandwidth pool based on the size of file segments on each server. For example, if two segments of a file, being 1GB and 2GB respectively, reside on BB1 and BB2, a read request with the QoS of 300MB/s will be divided into 100MB/s and 200MB/s. Finally, these two loads will be inserted into the load lists on BB1 and BB2. One downside is that if BB1 or BB2 is busy, users may only receive the maximum available bandwidth.

Currently, our priority is to fully utilize the bandwidth resources on loaded BB nodes to reduce bandwidth fragmentation. Besides, this approach allows us to free up more empty BB nodes to serve applications with larger bandwidth requirements. It is noted that the scheduling policy can also be changed easily to meet other objectives, such as balancing SSD wear by distributing I/O loads as evenly as possible across all BB nodes.

E. QoS Controller

The QoS Controller resides at the server side of the BB file system, which is located on each BB node. Each BB node has its own QoS Controller. It is responsible for enforcing the scheduling of concurrent I/O requests according to the bandwidth requirements of the processes that launch these I/O requests. As shown in Fig. 6, after the BB scheduler synchronizes a bandwidth request with a filesystem server, a QoS object is created using the global unique id of a process along with its desired bandwidth b' . The global unique id is generated from the `ip_address` of a compute node along with the rank of a process. When receiving an I/O request, the QoS controller first checks if its QoS object exists. If it exists, the I/O request will be tagged according to its QoS object. Based on the principle of the mClock algorithm, a request should have three tags: `rTag` for reservation, `lTag` for limitation, and `pTag` for proportion. Detailed description of mClock can be found in [24].

To ensure an accurate bandwidth guarantee, both the lower-bound and upper-bound bandwidth are set to b' . If users or

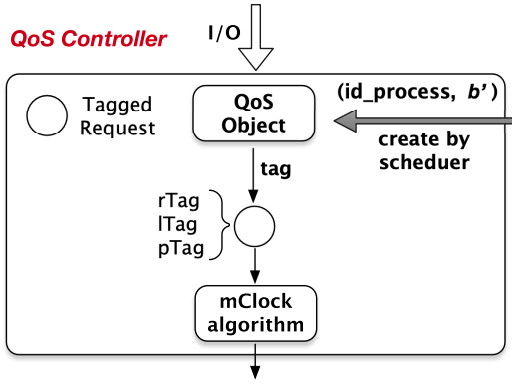


Fig. 6. Workflow of QoS Controller .

administrators want to make full use of the device bandwidth, they can set the upper-bound bandwidth to a high value. But in this case, the proportion bandwidth should also be configured to make sure the bandwidth is shared as they wish. In fact, we do not recommend this type of usage because it will cause the bandwidth resources out of control as the load lists maintained by the bandwidth pool will no longer reflect the actual loads.

F. Discussion

Supporting different HPC I/O patterns. In HPC, N-N, N-M, and N-1 are the three prevalent I/O patterns. RuYi supports all three patterns through its process-to-BB mapping that guarantees process-level QoS. Specifically, RuYi handles the BB allocation only for the processes that launch I/O operations, not for all processes. When using RuYi, it is important for users to understand the I/O pattern of their application and call RuYi interfaces only in I/O processes. However, applications using the 1-1 pattern (only one process writes one file) may have inherent performance limitations, as they can only obtain bandwidth resources from a single SSD. Should an 1-1 application submit a high bandwidth request, there is a significant chance it will remain unfulfilled. Therefore, users would be better not to use 1-1 pattern.

Supporting other performance metrics. At present, RuYi only provides bandwidth QoS guarantees for read and write operations, which cater to most usage scenarios. Another important metric is latency which is important for latency-sensitive workloads such as database query services. To provide latency guarantee, the bandwidth tag needs to be replaced by latency tag so that the mClock algorithm can prioritize the I/O requests demanding lower latency. But constructing a latency pool needs to take more hardware-level details into consideration and can be left for future work.

Handling metadata-intensive applications. When there are a mix of metadata-intensive and bandwidth-intensive applications, RuYi follows a straightforward strategy that prioritizes metadata operations over bandwidth-intensive operations during I/O scheduling. As HadaFS adopts the decentralized metadata management that hashes the metadata to different BB nodes, the I/O pressure caused by metadata-intensive applications will be dispersed across BB nodes, which mitigates the

impact on the bandwidth-intensive applications. Incorporating the metadata QoS management into an overall QoS framework has the potential to further improve the overall performance when dealing with a mix of metadata-intensive and bandwidth-intensive applications. However, this will be a very challenging task. We plan to address this in our future work.

Supporting big data & AI applications. In modern cloud platforms, users tend to place the data sets of big data and AI applications in shared SSD pools to accelerate I/O performance [28]. In this case, I/O QoS is also important. As big data or AI applications also use concurrent threads/processes to raise I/Os, e.g., BlockManager in Spark [29] and Dataloader in Pytorch [30], we believe that our fine-grained resource allocation method can benefit these applications and help saving SSD resources as well.

Usability. Currently, the (o, r, b) tuple for each process-level I/O load is pre-determined. Users can configure these three parameters based on their codes or through I/O profile tools. In particular, the operation type and request size are the parameters of the I/O functions called by the programmers, which are easy to determine. The third I/O feature, desired bandwidth, can be estimated by running a small-scale benchmarking code extracted from the corresponding I/O phase of an application (e.g., a representative portion of the application code running with a single process), which is also a common approach in performance modelling of HPC applications. Automatically obtaining these parameters would be another interesting but challenging task, which can further improve the usability of RuYi. We will leave this to the future work.

Limitation. Although the user-level interface of RuYi can provide powerful QoS control for processes, it requires modifications of the source code (e.g., calling the QoS_require() interface to specify the bandwidth requirements). Currently, on the Sunway system, almost all applications that want to use BB have to replace their I/O interfaces with the HadaFS I/O interfaces. Therefore, for these applications, it is straightforward to change their I/O interfaces to the corresponding RuYi interfaces. As for those applications whose source code cannot be modified, they can use the RuYi management interface to require bandwidth resources. However, the required resources will be limited to the default BB nodes that are bound to the compute nodes of the applications.

G. Implementation

RuYi is built on a novel BB file system named HadaFS. We mainly modify its client-side interfaces to enable users to specify their QoS requirements. We also modify its server-side I/O scheduling modules to integrate the mClock algorithm. The manager server is implemented using python and the communication between components is achieved using gRPC. As gRPC officially do not provide C interfaces, we use an open-source gRPC-C [31] to implement the filesystem side communication functions.

The implementation of map_to interface in Fig. 5 is dependent on the specific platform. RuYi enhances the HadaFS mount interface to dynamically establish a socket/ib connection

between a process and a BB node, and to attach a process id to the connection. This allows the filesystem server to identify I/Os from different processes, which is crucial for QoS control.

IV. EVALUATION

In this section, we mainly answer the following five questions:

- What is the end-to-end bandwidth control accuracy of RuYi on a large scale Buffer Buffer? (Section IV-B)
- Can the process-to-BB mapping strategy effectively improve the resource utilization? (Section IV-C)
- Can RuYi accurately predict the available resources in complex I/O environments? (Section IV-D)
- Can RuYi deal with inter-application interference? (Section IV-E)
- What is the overhead of RuYi? Can it be deployed on BB of exascale HPC systems? (Section IV-F)

A. Experimental Setup

We conducted evaluations of RuYi on a sub-system of the leading supercomputer Sunway. The sub-system consists of 4000 compute nodes and 200 BB nodes, while the full system consists of 100,000 compute nodes and 600 BB nodes. Each compute node at most raises 6 I/O processes, and each BB node is equipped with two 3TB SSDs, with one BB node serving as the manager server. Currently, HadaFS will flush data in a BB node to the underlying PFS when the data volume in the BB node reaches a certain threshold (which is set to be 70% by default).

In our evaluation tests, we use four I/O benchmarks, which are HACC-IO [32], WRF [33], VPIC-IO [34], and IOR [35]. The first three benchmarks are I/O kernels of real-world applications, while the last one is a microbenchmark commonly used in HPC I/O test. HACC (Hardware/Hybrid Accelerated Cosmology Code) is an application based on N-body simulation with N-N access pattern. In the checkpoint phase and restart phase, each process conducts 9 write/read operations whose I/O sizes are mostly $\text{num_particles} * 4$ bytes. As the HACC is consistently configured with particle counts in the range of millions [20], [36], the I/O size of each I/O operation is several mega bytes. In this evaluation, we configure the number of particles to one million, so that the I/O size of HACC is 4MB. WRF (Weather Research and Forecasting) is also an N-N application, but most of its I/Os is smaller than 1KB [37]. We choose these two applications to represent N-N applications with large and small request sizes, respectively. For N-1 applications, we choose VPIC (Vector Particle-in-cell) whose write operations are mostly 4MB.

B. End-to-End Bandwidth Control Accuracy

QoS control accuracy is a fundamental metric when assessing a QoS system. We evaluate the QoS control accuracy of RuYi on the sub-system of the leading supercomputer Sunway to test whether the QoS function works well in large scale BB in competing scenarios. We concurrently running 5 I/O jobs (HACC-IO, WRF and three IOR jobs) with varying I/O characteristics

TABLE II
END-TO-END BANDWIDTH CONTROL ACCURACY IN THE SCALE OF 4000 COMPUTE NODES AND 200 BB NODES

	Request Size	Write		Read	
		Desired/Obtained (GB/s)	Accuracy	Desired/Obtained (GB/s)	Accuracy
WRF	1KB	9.6/9.32	97.12%	9.6/9.45	98.45%
IOR1	64KB	19.2/18.99	98.93%	19.2/19.01	99.01%
IOR2	128KB	28.8/28.29	98.22%	28.8/28.45	98.79%
IOR3	512KB	38.4/37.77	98.35%	38.4/37.87	98.62%
HACC-IO	4MB	48.0/46.95	97.81%	48/46.81	97.53%

and bandwidth requirements. As VPIC-IO is a N-1 application and it can not have a bandwidth requirement larger than the performance of a single SSD, it is excluded from this large scale test.

Each I/O job runs on 800 compute nodes, with 6 processes per node, and has request sizes ranging from 1KB to 4MB. The desired bandwidth of these 5 jobs ranges from 9.6GB/s to 48GB/s in increments of 9.6GB/s, with each process having bandwidth requirements of 2MB/s, 4MB/s, 6MB/s, 8MB/s, and 10MB/s. The accuracy of both write and read operations is tested. Before testing, the LOESS curve is fitted over 200 BB nodes of Sunway like Fig. 4 to predict available bandwidth of BB nodes. As the prediction is not entirely accurate, we adjust the prediction results downwards by 20MB/s to ensure resource availability with little utilization sacrifice. The detailed analysis of the bandwidth prediction can be found in Section IV-D.

The bandwidth control accuracy is defined as $\frac{\text{obtained bandwidth}}{\text{desired bandwidth}} * 100\%$. As shown in Table II, the accuracy is higher than 97% for both read and write operations of all the 5 I/O jobs. These results indicates that RuYi effectively controls the bandwidth allocation. The little control error is mainly due to fluctuations in the performance of storage devices and is considered acceptable. With each job having 4800 processes, the absolute error of each I/O process is only tens of KB/s and can be disregarded. The results also show that the request size does not affect the control accuracy, proving that RuYi can ensure resource availability for both small and large requests. Additionally, it is observed that the actual bandwidth is always lower than the desired bandwidth. This is a result of the mClock algorithm, which prevents excessive resource allocation and ensures each job is assigned the appropriate amount of bandwidth.

In this test, WRF uses 80 SSDs to fulfill its bandwidth requirement as its request size is very small and consumes more bandwidth resources for the same amount of bandwidth requirement. The other four jobs use 12, 14, 15 and 17 SSDs, respectively. By analysing the manager trace file, we find that only 4 SSDs serve the I/O processes from two jobs while most of them serve a single job. This is because the large amount of I/O processes of each job makes the bandwidth requirement of each I/O process is too small to produce bandwidth fragmentation, and only the last SSD allocated for a job can have available bandwidth for another job. Furthermore, as these jobs are too large scale, there is hardly any utilization improvement in this evaluation, detailed analysis is presented in Section IV-C.

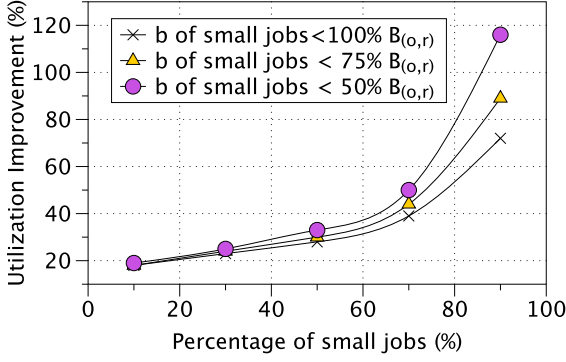


Fig. 7. RuYi improves resource utilization by 18% to 116% compared to the allocation method without QoS support and process-to-BB mapping. When the percentage of small jobs increases and their bandwidth requirements decrease, RuYi has greater potential to mitigate resource fragmentation and achieve a higher level of utilization improvement.

C. Resource Utilization Improvement

In this section, we assess whether RuYi is capable of improving resource utilization through process-to-BB mapping. To evaluate RuYi, we create different scenarios by varying three parameters. First, we increase the number of concurrent jobs from 10 to 100 with a step size of 10. Second, we change the proportion of small jobs from 10% to 90% with a step size of 20%. Small jobs are those jobs whose bandwidth requirements are less than the maximum capacity of a single SSD, i.e., $b < B_{(o,r)}$, while large jobs can request any amount of bandwidth resources, as long as the total bandwidth requirements of all the jobs do not exceed the total bandwidth capacity of the 200 BB nodes (400 SSDs). Third, we vary the maximum bandwidth requirements of small jobs from $50\%B_{(o,r)}$ to $100\%B_{(o,r)}$. This gives us a total of $10 \times 5 \times 3 = 150$ parameter combinations. For each combination, we randomly generate 10 sets of jobs and concurrently raise IOR jobs according to configurations. After a set of jobs finishes, we record the largest number of SSDs involved during runtime.

The baseline system refers to a BB resource allocation method without the QoS-aware ability and the process-to-BB mapping. We implement the baseline by simulating DFRA[11], a SOTA BB allocation method implemented on top of HadaFS (BB file system deployed on Sunway). It relies on pre-running an application multiple times to determine the number of BB nodes that can provide the application with the maximum bandwidth. Moreover, to avoid inter-application interference, DFRA tends to allocate dedicated BB nodes to different jobs.

We define the utilization improvement as $(\frac{N_{baseline}}{N_{RuYi}} - 1) \times 100\%$ to measure the advantages of RuYi, where $N_{baseline}$ refers to the number of BB nodes required to satisfy the bandwidth requirements of a set of jobs using the baseline method, and N_{RuYi} refers to the number of BB nodes used by RuYi.

As shown in Fig. 7, RuYi improves resource utilization by 18% to 116% in comparison to the baseline method. The improvement is made by the combined effort of reliable QoS and process-to-BB mapping adopted by RuYi, which allows I/O processes with much smaller bandwidth requirements to

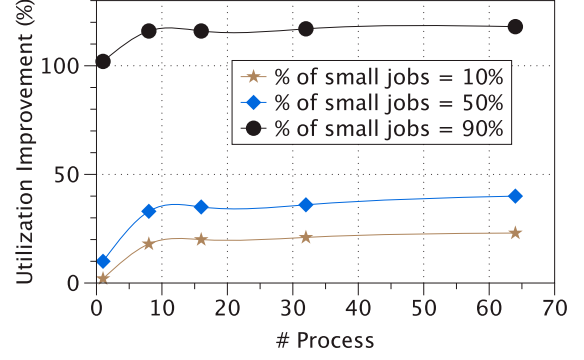


Fig. 8. Further analysis of utilization improvement by varying the number of processes. The maximum bandwidth requirements of small jobs is $50\%B_{(o,r)}$.

utilize the fragmented bandwidth resources and share BB without interference. It is noteworthy that RuYi performs better in scenarios where small jobs have a higher percentage and lower bandwidth requirements as this offers RuYi more opportunities to reduce resource fragmentation and achieve higher utilization improvement. In addition, the evaluation results with different numbers of concurrent jobs did not show significant differences, so they were not included in the figure.

We further analyze the case where the maximum bandwidth requirements of small jobs is $50\%B_{(o,r)}$ (purple curve in Fig. 7) to understand the impact of the number of processes on the improvement of resource utilization. As shown in Fig. 8, we varied the number of IOR processes from 1 to 64. Experimental results indicate that when the number of processes is 8, RuYi has achieved nearly maximum utilization improvement. This occurs because with 8 processes, the bandwidth requirements per process are small enough to utilize the bandwidth fragments in individual BB nodes effectively with our process-to-BB mapping.

D. Accuracy of Predicting Available Bandwidth

To evaluate the prediction accuracy, we generated a test data set of 10,000 samples. Additionally, we selected two well-established learning-based methods, XGBoost [38] and RF(Random Forest) [39], to compare with our method.

Generating test data set. The format of a sample is outlined in Table I. Generating a sample involves two steps. First, we randomly generate a background load list with different (o, r, b) tuples and start IOR jobs based on these tuples. As the cluster testbed has 6 compute nodes, it is important to ensure that the generated load list does not exceed 5, so that there is space for a separate job to be tested. Second, we run a separate IOR job with a random operation type o and a request size r on a spare compute node, and collect its bandwidth result b_r as the output. It is noted that for the first step, we ensure the total bandwidth requirement of all the (o, r, b) tuples does not exceed the maximum bandwidth of a single SSD. Furthermore, the IOR jobs in the background load list are set with a lower and upper-bound bandwidth to ensure stable and accurate bandwidth consumption, while the separate IOR job maximizes the upper-bound bandwidth to obtain all available bandwidth.

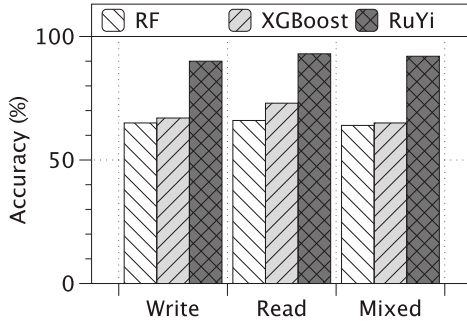


Fig. 9. Accuracy of predicting available bandwidth resources. RuYi outperforms two learning-based methods and achieves 92% in read/write mixed scenarios.

As comparison methods, XGBoost and RF are two widely used learning-based approaches used in SSD performance modeling that have demonstrated good results [40], [41]. For the convenience of training, load lists that are shorter than 5 elements are padded with zeros to maintain a consistent length of 5. As a result, each sample will have $2 + 3 \times 5 = 17$ input features and 1 output feature. It should be noted that padding does not affect the quality of the training samples, as the (0,0,0) tuple has its practical meaning as well; it represents a write operation with no bandwidth requirement. To achieve the best training results, we use RandomizedSearchCV to find the optimal hyperparameters¹ for both XGBoost and RF. The ratio of training samples to test samples is set to 8:2.

We quantify the prediction accuracy using the formula N_t/N , where N_t refers to the number of test samples whose predicted values differ from actual values by no more than 20MB/s, and N refers to the total number of test samples. We evaluate the prediction accuracy in three scenarios: write-only, read-only, and read/write-mixed. Our results, shown in Fig. 9, demonstrate that RuYi achieves greater than 92% accuracy in all three scenarios, while the other two learning-based methods only achieve around 60% accuracy despite our efforts to improve training. This may be because these regression methods struggle to handle situations where features have interdependent relationships of multiplication or division. The addition of the $b_i/B_{(o_i, r_i)}$ feature even caused the accuracy of the two learning-based methods to drop to around 50%. It is noted that the accuracy in the write-only scenario is lower than in the read-only scenario. This is because write operations tend to cause more I/O fluctuations in SSDs [42], causing some samples themselves to have deviations. Such deviations also cause that in the worst case, the difference between the prediction value and the actual value can reach up to nearly 110MB/s. But in most cases, the prediction is accurate and the average difference is 18MB/s, which is totally acceptable.

¹XGBoost: eta=0.1, gamma=0, max_depth=18, min_child_weight=10, n_estimators=1400, objective=reg:squarederror, RF: criterion=mse, max_depth=15, min_impurity_decrease=0.05, min_samples_split=10, n_estimators=2000

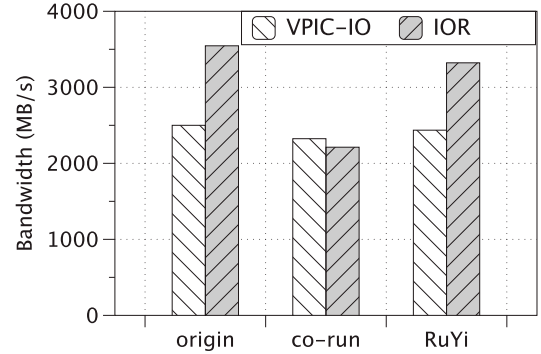


Fig. 10. Prevent interference between two jobs with N-N pattern (IOR) and N-1 pattern (VPIC-IO).

E. Dealing With Inter-Application Interference

N-1 pattern is recognized as a major cause of inter-application interference as it flushes I/Os to a single BB and consumes a large amount of storage resources. [7]. We simulate the interference scenario by setting up two jobs, one with the N-N pattern and the other with the N-1 pattern. We run the IOR benchmark with the N-N pattern while running the VPIC-IO benchmark with the N-1 pattern. Both jobs have the request size of 4MB and 12 processes running on 2 compute nodes. The bandwidth achieved by running VPIC-IO and IOR alone are approximately 2.5GB/s and 3.6GB/s, respectively. Two BB nodes are used in this test. The compute nodes of VPIC-IO bind to one BB node because of its N-1 pattern, while the compute nodes of IOR bind to two BB nodes for better performance.

As shown in Fig. 10, without the QoS support, IOR struggles to obtain the sufficient bandwidth on one BB node as the file system is overwhelmed by the requests from VPIC-IO. As a result, IOR only acquires 2.2GB/s (most from another BB node) bandwidth while VPIC-IO obtains 2.3GB/s. However, with the process-to-BB mapping, 12 IOR processes can be divided into two parts: 1 process binds to BB1 and 11 processes bind to BB2. The only I/O process that is bound to BB1 can obtain nearly 300MB/s bandwidth from BB1 thanks to RuYi. Experimental results show that with RuYi, VPIC-IO and IOR receive the bandwidths of 2.4GB/s and 3.3GB/s, respectively, which are close to the bandwidth that are achieved when they are running alone.

F. Overhead and Scalability

This section answers whether RuYi can be deployed on exascale BB with acceptable overhead. The overhead of RuYi can be broken down into three parts: scheduling overhead from the RuYi manager, synchronization overhead between the RuYi manager and filesystem servers, and the mClock algorithm overhead at the filesystem server-side. We evaluate these three overheads by simulating extreme conditions that could be encountered in exascale systems.

Scheduling overhead. As described in section III-C, the BB scheduler needs to first calculate the spare bandwidth capacity on all BB nodes and then select the appropriate ones to determine available BB nodes for all processes of a job. Thus, the

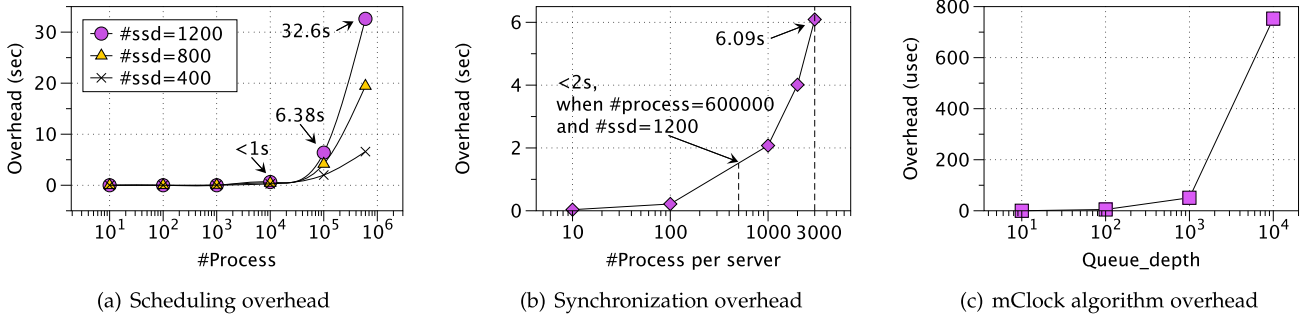


Fig. 11. The overhead of RuYi is projected to be acceptable in exascale systems. It takes at most 32.6 (for scheduling) + 2 (for synchronization) $\approx$ 35 seconds to initiate RuYi for the entire exascale Sunway with 600,000 I/O processes and 600 BB nodes (1200 SSDs). The overhead of mClock algorithm can be neglected.

scheduling overhead is directly proportional to the number of processes and BB nodes. To simulate the extreme scheduling pressure, we vary the number of processes from 10 to 600,000 and the number of BB nodes from 400 to 1,200. As shown in Fig. 11(a), the time spent on scheduling remains under 1 second when the number of processes is below 10,000. When the number of processes reaches 600,000, it takes 32.6 seconds to find proper targets from 1,200 SSDs. Despite this overhead, it remains acceptable because the I/O time of jobs with hundreds of thousands of processes is much longer than tens of seconds.

Synchronization overhead. The synchronization procedure in RuYi is aimed at creating Quality of Service (QoS) objects at the file system server and preparing the selected BB nodes for QoS control. As a result, the synchronization overhead is directly proportional to the number of QoS objects to be created. With an usual allocation granularity of 1MB/s and a maximum bandwidth of nearly 3GB/s per SSD in the Sunway BB, it is estimated that a BB node may create a maximum of 3,000 QoS objects for 3,000 processes. The results in Fig. 11(b) show that it takes 6.09 seconds to create 3000 QoS objects on a single BB node, which is a reasonable overhead. In typical scenarios, a BB node only needs to serve tens or hundreds of processes, which takes one or two seconds to prepare the QoS objects, which can be disregarded. It is noted that the number of BB nodes has a negligible impact on the synchronization overhead as the synchronization procedure can be executed concurrently on different BB nodes.

mClock algorithm overhead. The overhead of the mClock algorithm primarily stems from the complexity of its heap operations, such as insertion and removal, which are $O(\log n)$. As the heap size increases, the time required for these heap operations increases theoretically. Fig. 11(c) shows that when the queue depth reaches 10,000 under heavy I/O loads, the overhead of scheduling a request using the mClock algorithm reaches nearly 750 usec. Although this overhead is higher than the SSD latency (10usec), it is still considered negligible as the queuing time under such heavy I/O pressure can reach several seconds.

In conclusion, the results of our simulations indicate that RuYi can be deployed on exascale supercomputers with an acceptable overhead. In the most demanding scenario, it takes 32.6 (for scheduling) + 2 (for synchronization) $\approx$ 35 seconds to initiate RuYi for the entire exascale Sunway with 600,000 processes and 1200 SSDs. As for common applications that

have hundreds or thousands of processes, the one or two seconds setup overhead of RuYi can be neglected as the I/O time of these applications can reach up to tens or hundreds of seconds [7].

V. RELATED WORK

In terms of BB allocation, DataWarp [17] is the most classic and representative system. It provides users with two policies: bandwidth policy and interference policy. Under the bandwidth policy, jobs can access as many BB nodes as possible to maximize their obtained bandwidth. Conversely, under the interference policy, jobs receive fewer but dedicated BB nodes to avoid interference with others. These two policies represent the core principles of BB allocation: sharing and isolation. The sharing principle aims to achieve maximum bandwidth, while the isolation principle prioritizes the experience of individual applications. Based on these principles, we classify existing methods into two categories: interference-aware and QoS-aware.

Interference-aware BB allocation. In scenarios where BB is shared among applications, researchers have found that inter-application interference is a primary factor affecting performance. The work in [4] uses a global I/O queue to develop a scheduling policy with less I/O contention between high-level jobs. CARS [5] derives an I/O contention model to guide BB node allocation. The work in [6] properly dimensions the BB to ensure fairness between applications. DFRA [7] automatically adjusts the number of BB nodes allocated to a job based on both real-time monitoring results and past monitoring history. ThemisIO [8] and IO-SETS [9] propose various sharing policies to balance I/O cycles between applications.

QoS-aware BB allocation. This allocation method primarily focuses on accurately guaranteeing the desired bandwidth for applications. U-shape [13] uses dynamically determined service time windows to meet bandwidth requirements and is implemented in PVFS2. The work in [10] uses a token-bucket-like mechanism to limit RPC rates based on QoS rules and is implemented in Lustre. GIFT [11] considers the fairness of concurrent I/O processes and compensates slower I/O processes with more resources in subsequent I/O stages. SIREN [12] also enforces end-to-end QoS policies in BB, similar to RuYi.

Compute-to-BB mapping. To the best of our knowledge, existing research efforts, whether interference-aware or

QoS-aware, are based on the compute-to-BB mapping. They either attempt to map the compute nodes where applications reside to suitable BB nodes (such as CARS, DFRA) or use a default mapping from compute nodes to BB nodes and then make the best attempt to share the bandwidth of BB nodes among applications (such as TBF, ThemisIO). In existing work, a compute node (and the processes running on it) can be mapped to only one BB node.

RuYi is a QoS-aware allocation method with finer-grained process-to-BB mapping. The biggest difference between RuYi and existing works is that RuYi not only quantitatively profiles the bandwidth resources in BB to ensure resource availability but also refines the conventional BB architecture with a process-to-BB mapping to support fine-grained bandwidth allocation, thereby improving BB utilization. Additionally, as RuYi provides process-level QoS, it also guarantees fairness between concurrent I/O processes of a single application.

SSD performance modeling. Existing studies mainly model SSD performance through analytic modeling [43], simulation [44] or black-box approaches [45]. Recently, using learning-based approaches to model SSD performance become popular [40], [41] and has demonstrated promising results. While the output produced by these models is similar to that of our prediction method, the inputs they use are vastly different. Existing models primarily use hardware-level status or parameters of SSDs as inputs, while RuYi uses application-level I/O characteristics as inputs. Therefore, RuYi is more suitable to profile available bandwidth resources in HPC BB scenarios.

VI. CONCLUSION

In this paper, we examined critical issues in existing BB allocation methods, focusing mainly on the reliable bandwidth guarantees and the imperative for better resource utilization. To address these challenges, we combined the QoS control technique with a finer-grained process-to-BB mapping, and proposed a novel BB allocation method, RuYi. We presented the design of the QoS interface and the method of ensuring resource availability. We also proposed effective designs to integrate the process-to-BB mapping with the existing BB software stack. Evaluations on a leading exascale supercomputer verified the advantages of RuYi in bandwidth control and resource usage.

REFERENCES

- [1] N. Liu et al., "On the role of burst buffers in leadership-class storage systems," in *Proc. IEEE 28th Symp. Mass Storage Syst. Technol. (MSST)*, Piscataway, NJ, USA: IEEE Press, 2012, pp. 1–11.
- [2] K. Harms, H. S. Oral, S. Atchley, and S. S. Vazhkudai, "Impact of burst buffer architectures on application portability," Oak Ridge National Lab. (ORNL), Oak Ridge, TN (United States). Oak Ridge ..., Tech. Rep., 2016.
- [3] X. Shi, M. Li, W. Liu, H. Jin, C. Yu, and Y. Chen, "SSDUP: A traffic-aware ssd burst buffer for HPC systems," in *Proc. Int. Conf. Supercomputing*, 2017, pp. 1–10.
- [4] S. Thapaliya, P. Bangalore, J. Lofstead, K. Mohror, and A. Moody, "Managing I/O interference in a shared burst buffer system," in *Proc. 45th Int. Conf. Parallel Process. (ICPP)*, Piscataway, NJ, USA: IEEE Press, 2016, pp. 416–425.
- [5] W. Liang, Y. Chen, J. Liu, and H. An, "Contention-aware resource scheduling for burst buffer systems," in *Proc. 47th Int. Conf. Parallel Process. Companion*, 2018, pp. 1–7.
- [6] G. Aupy, O. Beaumont, and L. Eyraud-Dubois, "Sizing and partitioning strategies for burst-buffers to reduce IO contention," in *Proc. IEEE Int. Parallel Distrib. Process. Symp. (IPDPS)*, Piscataway, NJ, USA: IEEE Press, 2019, pp. 631–640.
- [7] X. Ji et al., "Automatic, application-aware I/O forwarding resource allocation," in *Proc. 17th USENIX Conf. File Storage Technol. (FAST)*, 2019, pp. 265–279.
- [8] E. Karrels et al., "Fine-grained policy-driven I/O sharing for burst buffers," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal.*, 2023, pp. 1–12.
- [9] F. Boito, G. Pallez, L. Teylo, and N. Vidal, "IO-SETS: Simple and efficient approaches for I/O bandwidth management," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 10, pp. 2783–2796, Oct. 2023.
- [10] Y. Qian et al., "A configurable rule based classful token bucket filter network request scheduler for the lustre file system," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal.*, 2017, pp. 1–12.
- [11] T. Patel, R. Garg, and D. Tiwari, "Gift: A coupon based throttle-and-reward mechanism for fair and efficient i/o bandwidth management on parallel storage systems," in *Proc. 18th USENIX Conf. File Storage Technologies (FAST 20)*, 2020, pp. 103–119.
- [12] S. Karki, B. Nguyen, and X. Zhang, "QOS support for scientific workflows using software-defined storage resource enclaves," in *Proc. IEEE Int. Parallel Distrib. Process. Symp. (IPDPS)*, Piscataway, NJ, USA: IEEE Press, 2018, pp. 95–104.
- [13] X. Zhang, K. Davis, and S. Jiang, "QOS support for end users of I/O-intensive applications using shared storage systems," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal.*, 2011, pp. 1–12.
- [14] D. Dai, Y. Chen, D. Kimpe, and R. Ross, "Two-choice randomized dynamic I/O scheduler for object storage systems," in *Proc. SC*, 2014, pp. 635–646.
- [15] X. He et al., "HadaFS: A file system bridging the local and shared burst buffer for exascale supercomputers," in *Proc. USENIX Conf. File Storage Technol. (FAST)*, 2023, pp. 215–230.
- [16] A. Kougkas, H. Devarajan, X.-H. Sun, and J. Lofstead, "Harmonia: An interference-aware dynamic I/O scheduler for shared non-volatile burst buffers," in *Proc. IEEE Int. Conf. Cluster Comput. (CLUSTER)*, Piscataway, NJ, USA: IEEE Press, 2018, pp. 290–301.
- [17] D. Henseler, B. Landsteiner, D. Petesch, C. Wright, and N. J. Wright, "Architecture and design of cray datawarp," in *Proc. Cray User Group (CUG)*, 2016, p. 11.
- [18] B. Yang et al., "End-to-end I/O monitoring on a leading supercomputer," in *Proc. 16th USENIX Symp. Netw. Syst. Design Implementation (NSDI)*, 2019, pp. 379–394.
- [19] J. L. Bez et al., "Access patterns and performance behaviors of multi-layer supercomputer I/O subsystems under production load," in *Proc. 31st Int. Symp. High-Perform. Parallel Distrib. Comput.*, 2022, pp. 43–55.
- [20] S. Snyder, P. Carns, K. Harms, R. Ross, G. K. Lockwood, and N. J. Wright, "Modular HPC I/O characterization with darshan," in *Proc. 5th Workshop Extreme-Scale Program. Tools (ESPT)*, Piscataway, NJ, USA: IEEE Press, 2016, pp. 9–17.
- [21] G. K. Lockwood, S. Snyder, T. Wang, S. Byna, P. Carns, and N. J. Wright, "A year in the life of a parallel file system," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal.*, Piscataway, NJ, USA: IEEE Press, 2018, pp. 931–943.
- [22] R. Gunasekaran, S. Oral, J. Hill, R. Miller, F. Wang, and D. Leverman, "Comparative I/O workload characterization of two leadership class storage clusters," in *Proc. 10th Parallel Data Storage Workshop*, 2015, pp. 31–36.
- [23] B. Zha and H. Shen, "Adaptively periodic i/o scheduling for concurrent HPC applications," *Electronics*, vol. 11, no. 9, 2022, Art. no. 1318.
- [24] A. Gulati, A. Merchant, and P. J. Varman, "mClock: Handling throughput variability for hypervisor IO scheduling," in *Proc. OSDI*, vol. 10, 2010, pp. 437–450.
- [25] P. C. Austin and E. W. Steyerberg, "Graphical assessment of internal and external calibration of logistic regression models by using loess smoothers," *Statist. Med.*, vol. 33, no. 3, pp. 517–535, 2014.
- [26] O. Yildiz, M. Dorier, S. Ibrahim, R. Ross, and G. Antoniu, "On the root causes of cross-application I/O interference in HPC storage systems," in *Proc. IEEE Int. Parallel Distrib. Process. Symp. (IPDPS)*, Piscataway, NJ, USA: IEEE Press, 2016, pp. 750–759.
- [27] M. Mubarak et al., "Quantifying I/O and communication traffic interference on dragonfly networks equipped with burst buffers," in *Proc. IEEE Int. Conf. Cluster Comput. (CLUSTER)*, Piscataway, NJ, USA: IEEE Press, 2017, pp. 204–215.

- [28] B. Reidys, J. Sun, A. Badam, S. Noghabi, and J. Huang, "{BlockFlex}: Enabling storage harvesting with {Software-Defined} flash in modern cloud platforms," in *Proc. 16th USENIX Symp. Oper. Syst. Design Implementation (OSDI)*, 2022, pp. 17–33.
- [29] "BlockManager in Spark," [Online]. Available: <https://spark.apache.org/docs/1.2.1/api/java/org/apache/spark/storage/BlockManager.html>
- [30] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, 2019, pp. 8026–8037.
- [31] "gRPC-C," [Online]. Available: <https://github.com/Juniper/grpc-c>
- [32] "HACC-IO," [Online]. Available: <https://github.com/glennklockwood/hacc-io>
- [33] "A description of the advanced research WRF version 3." [Online]. Available: <http://www2.mmm.ucar.edu/wrf/users/>
- [34] "VPIC-IO," [Online]. Available: <https://github.com/glennklockwood/vpic-io>
- [35] H. Shan and J. Shalf, "Using IOR to analyze the I/O performance for HPC platforms," Lawrence Berkeley National Lab. (LBNL), Berkeley, CA (United States), Tech. Rep., 2007.
- [36] D. Biswas, S. Neuwirth, A. K. Paul, and A. R. Butt, "An I/O performance evaluation of varying CephFS striping patterns," in *Proc. IEEE Int. Conf. Cluster Comput. Workshops (CLUSTER Workshops)*, Piscataway, NJ, USA: IEEE Press, 2023, pp. 25–31.
- [37] J. Layton, "IO pattern characterization of HPC applications," in *Proc. Int. Symp. High Perform. Comput. Syst. Appl. (HPCS)*, Berlin, Heidelberg: Springer, 2009, pp. 292–303.
- [38] T. Chen et al., "Xgboost: Extreme gradient boosting," *R Package Version 0.4-2*, vol. 1, no. 4, pp. 1–4, 2015.
- [39] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, pp. 5–32, 2001.
- [40] J.-E. Dartois, J. Boukhobza, A. Knefati, and O. Barais, "Investigating machine learning algorithms for modeling SSD I/O performance for container-based virtualization," *IEEE Trans. Cloud Comput.*, vol. 9, no. 3, pp. 1103–1116, Jul./Sep. 2019.
- [41] Z. Zhong, M. Xu, M. A. Rodriguez, C. Xu, and R. Buyya, "Machine learning-based orchestration of containers: A taxonomy and future directions," *ACM Comput. Surveys (CSUR)*, vol. 54, no. 10s, pp. 1–35, 2022.
- [42] L. Qian, B. Tang, B. Ye, J. Wu, X. Wang, and S. Lu, "Stabilizing and boosting I/O performance for file systems with journaling on nvme ssd," *Sci. China Inf. Sci.*, vol. 65, no. 3, 2022, Art. no. 132102.
- [43] P. Desnoyers, "Analytic modeling of SSD write performance," in *Proc. 5th Annu. Int. Syst. Storage Conf.*, 2012, pp. 1–10.
- [44] D. Gouk et al., "Amber*: Enabling precise full-system simulation with detailed modeling of all SSD resources," in *Proc. 51st Annu. IEEE/ACM Int. Symp. Microarchit. (MICRO)*, Piscataway, NJ, USA: IEEE Press, 2018, pp. 469–481.
- [45] H. H. Huang, S. Li, A. Szalay, and A. Terzis, "Performance modeling and analysis of flash-based storage devices," in *Proc. IEEE 27th Symp. Mass Storage Syst. Technol. (MSST)*, Piscataway, NJ, USA: IEEE Press, 2011, pp. 1–11.

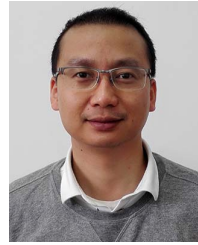


Yusheng Hua received the bachelor's degree in computer science and technology from Huazhong University of Science and Technology, China, in 2017. He is currently working toward the Ph.D. degree with the National Engineering Research Center for Big Data Technology and System/ Services Computing Technology and System Lab, Huazhong University of Science and Technology, China. His research interests focus on the HPC I/O, HPC QoS and parallel filesystem.



Xuanhua Shi (Senior Member, IEEE) received the Ph.D. degree in computer engineering from Huazhong University of Science and Technology, China, in 2005. He is a Professor with the National Engineering Research Center for Big Data Technology and System/Services Computing Technology and System Lab, Huazhong University of Science and Technology, China. Since 2006, he worked as an INRIA Postdoc with PARIS Team, Rennes, for one year. His research interests include cloud computing and big data processing. He published

over 100 peer-reviewed publications and received research support from a variety of governmental and industrial organizations, such as National Science Foundation of China, Ministry of Science and Technology, Ministry of Education, European Union, Alibaba, ByteDance, and Intel. He is a Senior Member of CCF.



Ligang He (Member, IEEE) is a Reader with the Department of Computer Science, University of Warwick, U.K. His research is mainly in the area of parallel and distributed computing, including distributed AI, Cluster/Cloud/Edge Computing as well as parallel and distributed data analytics. He has published more than 190 papers in the area, including top venues such as IEEE TRANSACTIONS ON COMPUTERS, TRANSACTIONS ON PARALLEL AND DISTRIBUTED SYSTEMS, TRANSACTIONS ON KNOWLEDGE AND DATA ENGINEERING, TRANSACTIONS ON CIRCUITS AND SYSTEMS FOR VIDEO TECHNOLOGY, NeurIPS, SC, EuroSys, IPDPS, ICPP, HPCA, VLDB, Micro, DAC, and so on.



Kang He received the bachelor's and master's degrees in computer science and technology from Huazhong University of Science and Technology, China, in 2020 and 2023, respectively. Currently, he is a Technical Staff with Alibaba. His research interests focus on the HPC IO, HPC QoS, and parallel filesystem.



Teng Zhang received the Ph.D. degree in computer science from Nanjing University, China, in 2019. Currently, he is an Assistant Professor with Huazhong University of Science and Technology. His major research interests include machine learning and data mining, especially on kernel methods and margin-based learning method. He has served as the Senior PC Member for IJCAI and ECAI, and a PC Member for ICML, NIPS, KDD, and AAAI.



Hai Jin received the Ph.D. degree in computer engineering from HUST, in 1994. In 1996, he was awarded a German Academic Exchange Service Fellowship to visit the Technical University of Chemnitz, Germany. He worked with The University of Hong Kong between 1998 and 2000, and as a Visiting Scholar with the University of Southern California between 1999 and 2000. He was awarded Excellent Youth Award from the National Science Foundation of China in 2001. He is a Chair Professor in computer science and engineering with Huazhong University of Science and Technology (HUST), China. He is a fellow of CCF and a Life Member of the ACM. His research interests include computer architecture, parallel and distributed computing, big data processing, data storage, and system security.



Yong Chen (Member, IEEE) is a Professor and the Director of the Data-Intensive Scalable Computing Laboratory with the Computer Science Department, Texas Tech University. His research interests include data-intensive computing, parallel and distributed computing, high-performance computing, and cloud computing.